

t

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

CÁTEDRA DE REDES DE COMPUTADORAS

TRABAJO PRÁCTICO Nº 7

Grupo: “Los simuLANdores”

Alumnos:

Barrio, Rafael

Chesta, Santino

Garay, Alexis Tomás

Guzmán Gonzalez, Pedro

Vera Gonzalez, Fernando Agustín

Martin, Agostina Rocio

Zucchella Paz, Valentino

Profesor:

Henn Santiago Martin

Desafio / consignas

1) Averigüen su IP y su máscara (ipconfig / ip addr) y calculen la dirección de broadcast de la red. ¿Cuál es la dirección MAC de broadcast?

   La direccion MAC del broadcast es “`ff:ff:ff:ff:ff:ff`”

2) Abran Wireshark en la interfaz Wi-Fi y capturen al menos 2 minutos. Pueden llegar varios mensajes; no tienen porqué ser todos idénticos.
3) Analicen la estructura del mensaje. ¿Qué formato tiene? ¿Qué campos contiene y qué creen que representa cada uno? Con lo visto en la sección 1, propongan cómo hay que interpretar los datos.

El formato que tiene es un mensaje JSON, contiene 3 campos, el cual creemos que cada uno representa:

- SIZE: el tamaño de la matriz, en formato “filas” y “columnas”
- PL (payload) : los datos
- CHECK: bits de control de paridad

La forma en la que hay que interpretar los datos es por ejemplo, si SIZE = 8,7, entonces tendríamos 8 filas y 7 columnas en ASCII, pero en bits tendríamos 8 filas x 49 columnas. Tomamos a PL como bits, ya que la paridad se calcula sobre ese campo y lo cargamos en la matriz; al campo CHECK, como contiene los bits de control (paridad de la fila y columna). Al tener PL y CHECK podemos verificar la paridad par: recalcular la de cada fila y de cada columna, y compararlas con las recibidas. Si hay diferencias, podemos ubicar los errores y corregirlos

4) Verifiquen la integridad del mensaje con el código de la sección 1: recalculen las paridades y compáralas con las recibidas. Registren qué filas y qué columnas fallan (el síndrome).

Tomamos un paquete de los recibidos y procedemos a analizar su integridad. La matriz es:

```
1101000111010011101001110000111001101110100101111 | 0
0101111111100111011111110101111010011101011100010 | 1
1100101010111011000111101111110110101011111110011 | 1
1101000110111111100101110100111001101011111110001 | 1
0111000111011111001111001011111101010010101110111 | 1
1010000110000101001000111111110011011001011100001 | 1
1110100111010111100101000101011110111100111101000 | 0
1100001111001011001010100000010000001000000100000 | 1
-----------------------------------------------
```

```
01101111111101100111001011110011000011001100100001
```

Se recalculo y comparo con lo obtenido, los resultados son:

Fila	Cantidad de unos	Recibida	Estado (✘ presenta falla)
1	28	0	✔
2	33	1	✔
3	33	1	✔
4	31	1	✔
5	32	1	✘
6	24	1	✘
7	29	0	✘
8	15	1	✔

Columna	Cantidad de unos	Recibida	Estado (✘ presenta falla)
9	8	1	✘
17	3	0	✘
23	6	1	✘

5) Si hay errores, ubíquenlos y corríjanlos. Para cada bit corregido indiquen fila, columna, valor recibido y valor corregido. Si recibieron más de un mensaje distinto, analicen cada uno por separado.

   Realizamos 2 tablas para resumir la corrección del paquete:

Fila	Cantidad de unos	Calculada	Recibida	Estado (✘ presenta falla)	Calculada tras corregir
1	28	0	0	✔	0
2	33	1	1	✔	1
3	33	1	1	✔	1

4	31	1	1	✔	1
5	32	0	1	✘	1
6	24	0	1	✘	1
7	29	1	0	✘	0
8	15	1	1	✔	1

Columna	Cantidad de unos	Calculada	Recibida	Estado (✘ presenta falla)	Calculada tras corregir
9	8	0	1	✘	1
17	3	1	0	✘	0
23	6	0	1	✘	1

La matriz corregida es:

```
11010001110100111010011100001110011011101001011111 | 0
01011111111001110111111101011110100111010111000010 | 1
11001010101110110001111011111101101010111111110011 | 1
11010001101111111001011101001110011010111111110001 | 1
01110001010111110011110010111111010100101011101111 | 1
10100001100001011010001111111110011011001011100001 | 1
11101001110101111001011001010111101111001111101000 | 0
11000011110010110010101000000100000010000000100000 | 1
--------------------------------------------------
01101111111101100111001011110011000011001100100001
```

6) Construyan un archivo de texto en el formato de mensaje.txt con los datos ya corregidos y decodifíquenlo con decodificar_mensaje.py (o el cuaderno). Si todo salió bien, el mensaje les va a indicar cómo seguir. Ojo: un mensaje que "se lee bien" no es necesariamente un mensaje correcto.

En el repositorio incluimos scripts de python los cuales desde una captura de wireshark ejecutaban la detección de paquetes y corrección de los mismos, hay incluidos en el repositorio archivos .md explicando como usar los scripts

La explicación corta es, capturar por wireshark, guardar la captura, luego ejecutar el script python pasandole la captura como parámetro al comando y esto genera en

una carpeta un informe con el paquete analizando, los bits detectados erróneos y todas las posibles correcciones, permitiéndonos analizar estas correcciones y elegir la correcta.

> El mensaje decodificado contiene el siguiente link: https://youtube.com/shorts/q8WgKzJwPa4?feature=share

Reflexiones e investigacion

1) ¿Qué hubiera pasado si el mensaje tenía errores en dos bits de filas y columnas distintas? ¿Y si se dañaba un bit de los de control?

El método de paridad bidimensional detecta ambas filas y columnas afectadas, pero no permite identificar los bits erróneos, ya que existen varias combinaciones posibles de intersecciones. En el contexto del TP nosotros recreamos todas las posibles combinaciones y elegimos la que efectivamente tuviese un link válido.

Si se daña un bit de control de una fila, aparecerá una discrepancia únicamente en la paridad de esa fila, de forma analoga con los bits de control de las columnas. Esto permite sospechar que el error está en el propio bit de control, aunque no garantiza identificarlo en todos los escenarios con múltiples errores.

2) Si el mensaje es texto en castellano, muchas veces un error "se adivina" por contexto (Hola mondo). ¿Sirve eso para una contraseña, un número de cuenta o una dirección web? ¿Qué aporta el código frente a "leer y adivinar"?

La corrección por contexto puede funcionar en textos escritos en lenguaje natural, pero no resulta confiable para contraseñas, números de cuenta o direcciones web, donde modificar un solo carácter puede cambiar completamente su significado.

Los códigos de detección y corrección de errores aportan un mecanismo matemático independiente del contenido del mensaje, que permite verificar su integridad y, **dentro de sus capacidades**, recuperar los datos originales sin necesidad de interpretar su significado.

3) ¿Pudieron confirmar que su corrección es correcta? ¿Cómo?

Pudimos pero no únicamente con las paridades, ya que en algunos casos existían múltiples combinaciones de bits que satisfacían las condiciones de paridad.

Por ello, exploramos las distintas correcciones posibles y decodificamos los mensajes resultantes. Algunos conducían a videos que no corresponden al desafío, como una escena de hackeo, una animación de un unicornio, una entrevista a Messi y un penal intrascendente. Otros generaban enlaces inválidos.

Finalmente, seleccionamos el enlace que correspondía al desafío, utilizando la validez y el contenido del recurso como criterios adicionales de verificación.

Esto demuestra que satisfacer las condiciones de paridad no garantiza haber recuperado el mensaje original, especialmente cuando existen múltiples errores. La confirmación requirió información externa al código de corrección.

4) Investigar y describir con el mayor detalle posible cómo funciona la técnica de corrección Hamming (7,4) y por qué es superior a la demostración didáctica que hicimos en este Laboratorio. Luego, aplicar la técnica sobre la carga útil obtenida en el desafío y elaborar un paquete de datos que podrían transmitir para que, en caso de corrupción, pueda ser recuperado.

El código Hamming (7,4) es una técnica de corrección de errores que transforma bloques de 4 bits de información en bloques de 7 bits, incorporando 3 bits de paridad.

Los bits se organizan de la siguiente manera:

Posición	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Contenido	P1	P2	D1	P4	D2	D3	D4

Los bits de paridad ocupan posiciones que son potencias de 2 (1, 2 y 4). Utilizando paridad par, se calculan mediante operaciones XOR:

$$P1 = D1 \oplus D2 \oplus D4$$

$$P2 = D1 \oplus D3 \oplus D4$$

$$P3 = D2 \oplus D3 \oplus D4$$

En recepción se vuelven a verificar las paridades:

- S1 comprueba las posiciones 1, 3, 5 y 7.
- S2 comprueba las posiciones 2, 3, 6 y 7.
- S4 comprueba las posiciones 4, 5, 6 y 7.

El síndrome se obtiene como $S = (S4\,S2\,S1)_2$. Si vale cero, no se detectan errores.

Si es distinto de cero, indica la posición del bit que debe invertirse, suponiendo que ocurrió un único error.

Por ejemplo, si el síndrome es 101, su valor decimal es 5, por lo que se corrige el bit ubicado en la posición 5.

Ventajas frente al método del laboratorio

Hamming (7,4) permite corregir automáticamente un error de un bit por bloque, incluso si afecta a un bit de control. Además, utiliza un síndrome que identifica directamente la posición del error.

En comparación, la paridad bidimensional del laboratorio puede corregir un error aislado y detectar ciertas combinaciones de errores múltiples, pero puede presentar ambigüedades al intentar corregirlos.

Hamming (7,4) tiene distancia mínima 3, por lo que puede corregir un error o detectar hasta dos errores por bloque, pero no ambas cosas simultáneamente de forma garantizada. Para distinguir de manera confiable errores simples de dobles se utiliza una variante extendida, conocida como SECDED.

Aplicación sobre la carga útil del desafío

Tomamos los 392 bits de la matriz corregida del punto 5, leyendo las filas de izquierda a derecha y de arriba hacia abajo, sin incluir los bits de control.

Aplicamos Hamming (7,4) a cada grupo de cuatro bits:

Concepto	Resultado
Datos originales	392 bits
Bloques de 4 bits	98
Bits de paridad agregados	294
Tamaño codificado	686 bits
Sobrecarga	75%

Por ejemplo, los primeros cuatro bits de la carga útil son 1101, que se codifican como 1010101.

El siguiente es el paquete codificado completo, expresado en hexadecimal para facilitar su representación. Se conserva el tamaño de 686 bits, incluyendo los dos ceros iniciales necesarios para completar la representación hexadecimal.

Carga útil codificada (Hamming 7,4)

2ae9ab0ed0f005b316323ed7fff0d9ffb5fe61667c19aab459e0f67ff35afe587da863ffaa
2c65e551fff48fd297fc378cffa554cd9d500f1555d3fcb6678cf80fb43d2964a65296ff0e
ae9e03cce64b30070d201500

Para transmitirlo, se debe indicar que el contenido está codificado con Hamming (7,4), que contiene 98 bloques y que los primeros dos bits de relleno de la representación hexadecimal no pertenecen al código.

De esta manera, si durante la transmisión se altera un único bit de cualquiera de los bloques de 7 bits, el receptor podrá localizarlo y corregirlo mediante el síndrome, recuperando los cuatro bits originales.

Conclusión: Hamming (7,4) proporciona una corrección sistemática y automática de errores simples, a cambio de introducir redundancia. No garantiza recuperar la información cuando ocurren dos o más errores dentro del mismo bloque.